



Section 1 / 13

Buffer Overflows Overview

Buffer overflows have become less common in today's world as modern compilers have built in memory-protections that make it difficult for memory corruption bugs to occur accidentally. That being said, languages like C are not going to go away anytime soon and they are predominant in embedded software and IOT (Internet of Things). One of my favorite somewhat recent Buffer Overflows was [CVE-2021-3156](#), which was a Heap-Based Buffer Overflow in sudo.

These attacks aren't limited to binaries, a large number of buffer overflows occur in web applications, especially embedded devices which utilize custom web servers. A good example is [CVE-2017-12542](#) with HP iLO (Integrated Lights Out) Management devices. Just sending 29 characters in an HTTP Header parameter caused a buffer overflow which bypassed login. I like this example because there is no need for an actual payload that you'll read more about later since the system "[failed open](#)" upon reaching an error.

In short, buffer overflows are caused by incorrect program code, which cannot process too large amounts of data correctly by the CPU and can, therefore, manipulate the CPU's processing. Suppose too much data is written to a reserved memory `buffer` or `stack` that is not limited, for example. In that case, specific registers will be overwritten, which may allow code to be executed.

A buffer overflow can cause the program to crash, corrupt data, or **harm data structures** in the program's runtime. The last of these can overwrite the specific program's **return address** with arbitrary data, allowing an attacker to execute commands with the **privileges of the process** vulnerable to the buffer overflow by passing arbitrary machine code. This code is usually intended to give us more convenient access to the system to use it for our own purposes. Such buffer overflows in common servers, and Internet worms also exploit client software.

A particularly popular target on Unix systems is root access, which gives us all permissions to access the system. However, as is often misunderstood, this does not mean that a buffer overflow that "only" leads to the privileges of a standard user is harmless. Getting the coveted root access is often much easier if you already have user privileges.

Buffer overflows, in addition to programming carelessness, are mainly made possible by computer systems based on the Von-Neumann architecture.

The most significant cause of buffer overflows is the use of programming languages that do not automatically monitor limits of memory buffer or stack to prevent (stack-based) buffer overflow. These include the **C** and **C++** languages, which emphasize performance and do not require monitoring.

For this reason, developers are forced to define such areas in the programming code themselves, which increases vulnerability many times over. These areas are often left undefined for testing purposes or due to carelessness. Even if they were used for testing purposes, they might have been overlooked at the end of the development process.

However, not every application environment will likely exhibit a buffer overflow condition. For example, a stand-alone Java application is least likely compared to others because of how Java

handles memory management. Java uses a "garbage collection" technique to manage memory, which helps prevent buffer overflow conditions.

Section 1 / 13

 +20

Next

Completed 